

Universidad de Antioquia

Facultad de Ingeniería

Informática II

Momento II

Diseño y Análisis

Proyecto final: **Clavados en Ciudad Academia**

Integrantes:

Camilo Andrés Villanueva González

José Manuel Posada Londoño

Profesor:

Anibal Guerra

Medellín, Colombia

Mayo de 2026

1. Introducción

Este videojuego 2D desarrollado en C++ con Qt como *Clavados en Ciudad Academia*, inspirado en el deporte de los clavados y ambientado en un universo tecnológico y académico. Esta segunda entrega profundiza en el diseño lógico del sistema, las vistas de los niveles, las interacciones entre los elementos del juego, las físicas a implementar, los sprites propuestos y la arquitectura del agente inteligente.

En el Momento I se definió la idea general del videojuego: una estudiante-atleta llamada Mikoto Misaka que participa en competencias de clavados dentro de Ciudad Academia, donde las pruebas deportivas se ven modificadas por plataformas móviles, viento artificial, zonas especiales y sistemas inteligentes de supervisión.

El objetivo principal del Momento II es mostrarlo mas como una arquitectura de clases mas claras, justificables y adecuadas para ser implementadas mediante programación orientada a objetos, memoria dinámica, contenedores STL e interfaz gráfica en Qt.

2. Cambios y ampliaciones respecto al Momento I

La propuesta general del Momento I se mantiene: el juego conserva el deporte base de los clavados, el universo futurista de Ciudad Academia, la protagonista Mikoto Misaka, los dos niveles diferenciados y el dron de supervisión como agente autónomo. Sin embargo, para esta entrega se amplían varios aspectos de diseño:

- Se precisan las vistas de cada nivel y la forma en que el jugador interactúa con el entorno.
- Se definen con mayor claridad los objetos de juego: plataformas, anillos, obstáculos, zonas de viento, piscina y dron.
- Se especifican las físicas que se van a representar mediante modelos parametrizables.
- Se propone una estructura de clases para la capa lógica del videojuego.
- Se incorpora herencia propia en la jerarquía de niveles, entidades y modelos físicos.
- Se identifican los contenedores STL necesarios para almacenar niveles, obstáculos, físicas, intentos y registros del agente inteligente.
- Se detalla la arquitectura del agente autónomo mediante percepción, razonamiento, acción y aprendizaje.

3. Descripción general del videojuego

Clavados en Ciudad Academia es un videojuego 2D en el que el jugador controla a Mikoto Misaka durante pruebas deportivas de clavados. El objetivo principal es realizar saltos correctos, controlar la trayectoria durante la caída, ajustar la postura del cuerpo y lograr una entrada precisa al agua.

La dificultad del juego no depende únicamente de caer en la piscina, sino de adaptarse a condiciones variables del escenario. En cada nivel aparecen elementos que afectan el movimiento del personaje, como viento lateral, plataformas oscilantes, zonas de velocidad variable y obstáculos flotantes. Además, un dron de supervisión analiza el desempeño del jugador y modifica ciertas condiciones para mantener el reto.

El videojuego estará compuesto inicialmente por dos niveles:

- **Nivel 1: Piscina de entrenamiento.** Vista lateral fija, orientada al aprendizaje de las mecánicas básicas.
- **Nivel 2: Torre experimental.** Vista lateral con scroll vertical limitado, mayor altura, temporizador, obstáculos y agente inteligente con mayor intervención.

4. Nivel 1: Piscina de entrenamiento

4.1. Contexto narrativo

El primer nivel se desarrolla en la piscina principal de entrenamiento de Ciudad Academia. Este lugar funciona como una zona de clasificación inicial, donde la protagonista debe demostrar que tiene control suficiente sobre su salto antes de acceder a pruebas más complejas.

Aunque se trata de un escenario de entrenamiento, la piscina cuenta con tecnología experimental. Por esta razón, la plataforma puede moverse ligeramente y el sistema puede activar corrientes de viento artificial para alterar la trayectoria del salto.

4.2. Vista del nivel

El nivel utilizará una **vista lateral fija**. Esta vista permite observar claramente la plataforma, el salto, la trayectoria parabólica del personaje y la entrada al agua. Al no tener desplazamiento de cámara, el jugador puede concentrarse en aprender las mecánicas principales.

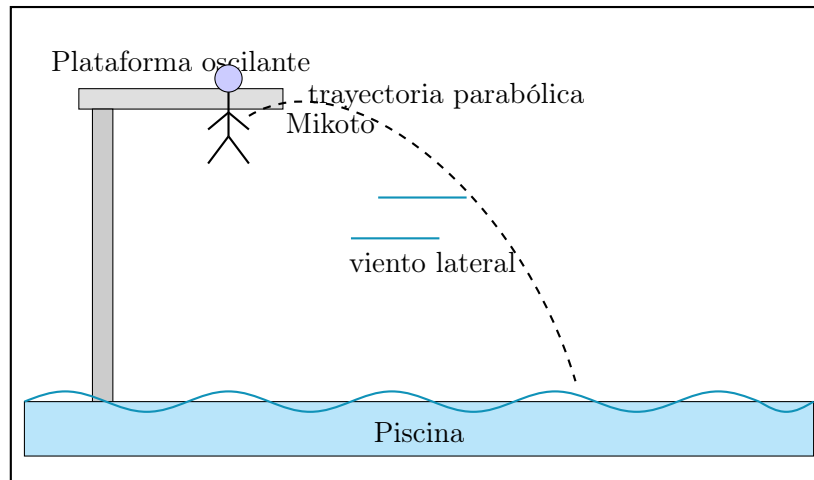


Figura 1: Dibujo informal de la vista lateral fija del Nivel 1.

4.3. Dinámica de juego

En este nivel, el jugador debe elegir el momento adecuado para saltar desde la plataforma. Después del salto, podrá corregir ligeramente la postura del personaje durante la caída. La calidad del intento se evaluará al llegar al agua.

La dinámica se organiza en las siguientes fases:

1. **Preparación:** Mikoto se encuentra sobre una plataforma que oscila suavemente.
2. **Impulso:** el jugador decide cuándo saltar. El momento del salto afecta la trayectoria inicial.
3. **Caída:** el personaje sigue una trayectoria parabólica y puede recibir viento lateral.
4. **Corrección:** el jugador ajusta la postura para mejorar la entrada al agua.
5. **Evaluación:** el sistema calcula el puntaje según la orientación, posición y estabilidad del salto.

4.4. Interacciones principales

Elemento	Interacción
Mikoto	Es controlada por el jugador. Puede saltar, girar, corregir postura y usar un impulso electromagnético limitado.
Plataforma oscilante	Cambia ligeramente el punto de partida del salto. Obliga a calcular el momento correcto.
Viento lateral	Aplica una fuerza horizontal que modifica la trayectoria durante la caída.
Piscina	Zona de llegada. Evalúa si la entrada fue correcta o defectuosa.
Sistema de puntaje	Calcula la calidad del clavado según postura, entrada y trayectoria.

Tabla 1: Interacciones principales del Nivel 1.

4.5. Físicas utilizadas en el Nivel 1

En este nivel se implementarán tres modelos físicos principales:

4.5.1. Movimiento parabólico

El salto se modela mediante una trayectoria parabólica. La posición del personaje se puede calcular como:

$$x(t) = x_0 + v_{0x}t$$

$$y(t) = y_0 + v_{0y}t + \frac{1}{2}gt^2$$

donde x_0 y y_0 representan la posición inicial, v_{0x} y v_{0y} son las componentes de velocidad inicial, g es la gravedad y t es el tiempo.

4.5.2. Movimiento oscilatorio de la plataforma

La plataforma puede moverse horizontalmente mediante una función oscilatoria:

$$x_p(t) = A \sin(\omega t + \phi)$$

donde A es la amplitud, ω es la frecuencia angular y ϕ es la fase inicial.

4.5.3. Viento lateral

El viento se modela como una aceleración horizontal adicional:

$$a_x = \frac{F_v}{m}$$

donde F_v representa la fuerza del viento y m la masa del personaje. Esta aceleración altera la componente horizontal de la velocidad durante la caída.

4.6. Objetivo del nivel

El objetivo del Nivel 1 es alcanzar una puntuación mínima en una cantidad limitada de intentos. El jugador debe demostrar que entiende las mecánicas básicas del salto, la caída y la entrada al agua.

4.7. Sprites propuestos para el Nivel 1

Para este nivel se utilizarán como mínimo los siguientes sprites:

- Mikoto en posición inicial.
- Mikoto saltando.
- Mikoto en caída.
- Mikoto entrando al agua.
- Plataforma oscilante.
- Efecto de viento lateral.
- Salpicadura al entrar al agua.



Figura 2: Sprites propuestos para el Nivel 1

5. Nivel 2: Torre experimental

5.1. Contexto narrativo

El segundo nivel ocurre en una torre experimental de inmersión avanzada. Esta instalación está diseñada para atletas con mayor experiencia y combina altura, obstáculos, zonas de viento, plataformas móviles y sistemas inteligentes de supervisión.

A diferencia del primer nivel, aquí el jugador no solo debe realizar una buena entrada al agua, sino también controlar su trayectoria durante un descenso más largo. Además, el nivel está limitado por tiempo, por lo que el jugador debe tomar decisiones rápidas.

5.2. Vista del nivel

El Nivel 2 utilizará una **vista lateral con scroll vertical limitado**. La cámara seguirá al personaje mientras cae desde la torre, generando una sensación de altura y movimiento. Esta vista se diferencia claramente de la vista fija del Nivel 1.

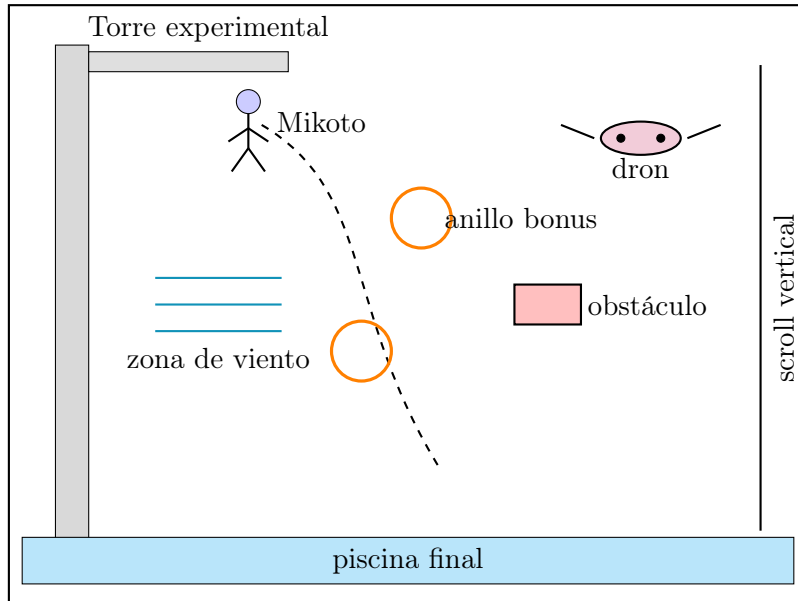


Figura 3: Dibujo informal de la vista lateral con scroll vertical del Nivel 2.

5.3. Dinámica de juego

El jugador inicia en la parte superior de la torre experimental. Después del salto, la cámara acompaña el descenso. Durante la caída aparecen anillos de bonificación, obstáculos flotantes, zonas de viento y variaciones de velocidad.

El nivel se desarrolla bajo presión de tiempo. Si el temporizador llega a cero antes de completar el recorrido, el intento se considera fallido.

Las fases principales son:

1. **Inicio en la torre:** el jugador se prepara para saltar desde una mayor altura.
2. **Descenso con scroll:** la cámara sigue al personaje mientras cae.
3. **Interacción con objetos:** el jugador puede atravesar anillos, evitar obstáculos y corregir su trayectoria.
4. **Intervención del dron:** el agente inteligente ajusta elementos del entorno.

5. **Entrada al agua:** el jugador debe llegar antes de que termine el tiempo y con una postura adecuada.

5.4. Interacciones principales

Elemento	Interacción
Mikoto	Controla postura, trayectoria e impulso electromagnético durante la caída.
Anillos de bonificación	Otorgan puntos adicionales si el jugador los atraviesa correctamente.
Obstáculos flotantes	Penalizan al jugador si ocurre una colisión.
Zonas de viento	Modifican la dirección horizontal del personaje.
Zonas de velocidad variable	Aumentan o disminuyen temporalmente la velocidad de caída.
Dron supervisor	Analiza el rendimiento del jugador y modifica algunos parámetros del escenario.
Temporizador	Limita el tiempo disponible para completar el nivel.

Tabla 2: Interacciones principales del Nivel 2.

5.5. Físicas utilizadas en el Nivel 2

5.5.1. Movimiento parabólico extendido

El salto conserva el modelo parabólico, pero con mayor tiempo de caída debido a la altura de la torre.

$$y(t) = y_0 + v_{0y}t + \frac{1}{2}gt^2$$

5.5.2. Corrientes laterales de aire

Al atravesar zonas de viento, el personaje recibe una aceleración horizontal temporal:

$$v_x(t + \Delta t) = v_x(t) + a_v \Delta t$$

donde a_v es la aceleración producida por el viento y Δt es el intervalo de actualización.

5.5.3. Zonas de velocidad variable

Algunas zonas modifican la velocidad vertical del personaje:

$$v'_y = kv_y$$

donde k es un factor de modificación. Si $k > 1$, la caída se acelera; si $0 < k < 1$, la caída se reduce temporalmente.

5.5.4. Oscilación de obstáculos o plataformas

Algunos elementos pueden moverse de forma periódica:

$$x_o(t) = x_{base} + A \sin(\omega t)$$

Esto permite que los obstáculos no sean completamente estáticos.

5.6. Objetivo del nivel

El objetivo del Nivel 2 es completar el recorrido antes de que termine el temporizador, atravesar la mayor cantidad posible de anillos de bonificación, evitar obstáculos y lograr una entrada correcta al agua.

5.7. Sprites propuestos para el Nivel 2

Para este nivel se utilizarán como mínimo los siguientes sprites:

- Mikoto en caída vertical.
- Mikoto usando impulso electromagnético.
- Dron supervisor.
- Anillo de bonificación.
- Obstáculo flotante.
- Zona de viento.
- Efecto de velocidad variable.

- Salpicadura final.



Figura 4: Sprites propuestos para el Nivel 2

6. Características del protagonista

Mikoto Misaka tendrá características que no serán únicamente narrativas, sino que afectarán directamente la jugabilidad.

6.1. Precisa

La precisión permite que el personaje tenga una mejor capacidad de corrección de postura durante la caída. En términos de juego, esto puede reflejarse en una mayor sensibilidad de control o en una reducción de penalización cuando la entrada al agua no es perfecta.

6.2. Impulsiva

La impulsividad permite realizar saltos más fuertes desde el inicio, pero aumenta el riesgo de error si el jugador no calcula bien el momento. Esta característica puede representarse mediante una velocidad inicial mayor, pero con menor margen de corrección.

6.3. Competitiva

La competitividad justifica la progresión de dificultad entre niveles. También puede relacionarse con bonificaciones por superar puntajes anteriores.

6.4. Afinidad electromagnética

La afinidad electromagnética permite usar un impulso limitado durante la caída. Este impulso puede corregir levemente la trayectoria o estabilizar el cuerpo, pero consume energía y requiere tiempo de recarga.

7. Dificultad del juego

La dificultad será parte del diseño del videojuego y no solo un aumento arbitrario de velocidad. Se propone manejar tres niveles de dificultad:

Dificultad	Características
Fácil	Menor intensidad de viento, plataformas más lentas, mayor tiempo disponible y menos obstáculos.
Normal	Parámetros equilibrados
Difícil	Mayor intensidad de viento, plataformas más rápidas, menor tiempo disponible y más obstáculos.

Tabla 3: Configuración propuesta de dificultad.

La dificultad se puede representar mediante una clase `Dificultad`, encargada de almacenar parámetros como:

- Intensidad del viento.
- Velocidad de plataformas.

- Cantidad de obstáculos.
- Tiempo límite del nivel.
- Puntuación mínima requerida.

8. Diseño de la capa lógica

La capa lógica del videojuego estará encargada de representar las reglas internas del sistema, sin depender directamente de los elementos gráficos de Qt. Esta separación permite que la lógica del juego sea más fácil de probar, modificar y justificar.

Las clases principales se organizan en los siguientes grupos:

- **Control general del juego:** Juego, Nivel, Temporizador, Dificultad.
- **Entidades del mundo:** Entidad, Personaje, Protagonista, Plataforma, Obstaculo, AnilloBonificacion, ZonaViento.
- **Físicas:** Fisica, FisicaParabolica, FisicaOscilatoria, FisicaViento, FisicaVelocidadVariable.
- **Agente inteligente:** AgenteInteligente, DronSupervisor, Percepcion, Razonamiento, AccionAgente, Aprendizaje.
- **Evaluación:** Puntaje, Intento.

9. Diagrama de clases de la capa lógica

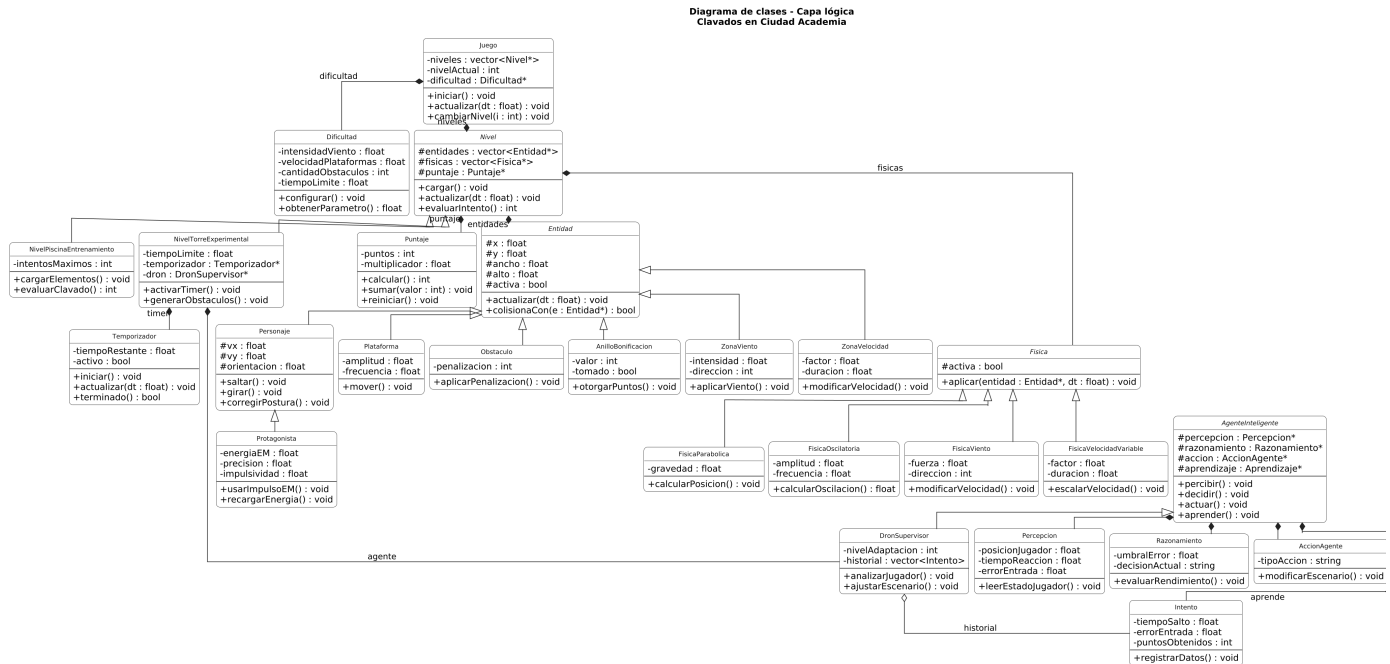


Figura 5: Diagrama de clases de la capa lógica del videojuego *Clavados en Ciudad Academia*.

10. Herencias

10.1. Herencia en niveles

La clase abstracta **Nivel** define atributos y métodos generales para cualquier nivel del juego. De ella heredan:

- **NivelPiscinaEntrenamiento**
- **NivelTorreExperimental**

Ambos niveles comparten funciones como cargar elementos, actualizar entidades y evaluar intentos, pero cada uno implementa su propia dinámica.

10.2. Herencia en entidades

La clase **Entidad** representa cualquier objeto que tenga posición, tamaño y estado dentro del juego. De ella heredan clases como:

- Personaje
- Plataforma
- Obstaculo
- AnilloBonificacion
- ZonaViento

Esto permite manejar varios objetos de forma uniforme dentro de un contenedor como `std::vector<Entidad*>`.

10.3. Herencia en físicas

La clase `Fisica` define una interfaz común para aplicar modelos físicos a las entidades. De ella heredan:

- `FisicaParabolica`
- `FisicaOscilatoria`
- `FisicaViento`
- `FisicaVelocidadVariable`

10.4. Herencia en agente inteligente

La clase `AgenteInteligente` define el comportamiento general de un agente con percepción, razonamiento, acción y aprendizaje. La clase `DronSupervisor` hereda de ella y adapta esa estructura al contexto del videojuego.

11. Contenedores STL propuestos

El proyecto usará contenedores STL para manejar colecciones de objetos y datos históricos. Su uso facilita la administración dinámica de elementos durante el juego.

Tabla 4: Contenedores STL propuestos para la capa lógica.

Contenedor	Uso propuesto
<code>std::vector<Nivel*></code>	Almacenar los niveles disponibles del juego.
<code>std::vector<Entidad*></code>	Guardar las entidades activas dentro de un nivel.
<code>std::vector<Obstaculo*></code>	Manejar obstáculos del Nivel 2.
<code>std::vector<AnilloBonificacion*></code>	Almacenar los anillos de bonificación.
<code>std::vector<Fisica*></code>	Guardar los modelos físicos activos en cada nivel.
<code>std::vector<Intento></code>	Registrar los intentos del jugador para evaluación y aprendizaje.
<code>std::vector<float></code>	Guardar errores recientes o datos numéricos del jugador.
<code>std::map<std::string,int></code>	Asociar parámetros o tendencias aprendidas por el agente inteligente.

El uso de punteros en algunos vectores permite trabajar con polimorfismo. Por ejemplo, almacenando objetos de varios tipos.

12. Arquitectura del agente inteligente

El agente inteligente será un **dron supervisor** que observa el desempeño del jugador y modifica ciertos elementos del escenario. Su objetivo no es jugar contra el usuario de forma compleja, sino adaptar la dificultad de manera sencilla y justificable.

El agente se compone de cuatro módulos principales:

- Percepción.
- Razonamiento.
- Acción.
- Aprendizaje.

12.1. Percepción

La percepción corresponde a la etapa en la que el dron recopila datos del entorno y del jugador. Los datos que puede observar son:

- Tiempo que tarda el jugador en saltar.
- Posición del personaje durante la caída.
- Cantidad de anillos atravesados.
- Colisiones con obstáculos.
- Calidad de entrada al agua.
- Uso del impulso electromagnético.

Estos datos se almacenan temporalmente para ser usados por el módulo de razonamiento.

12.2. Razonamiento

El razonamiento interpreta los datos obtenidos por la percepción. Por ejemplo, si el jugador falla varias veces por desviarse hacia la izquierda, el dron identifica una tendencia. Si el jugador obtiene puntajes muy altos repetidamente, el dron puede decidir aumentar un poco la dificultad.

Las posibles decisiones son:

- Mantener la dificultad actual.
- Reducir levemente la intensidad del viento.
- Aumentar la intensidad del viento.
- Mover obstáculos a zonas más retadoras.
- Activar anillos de bonificación para facilitar recuperación de puntos.

12.3. Acción

La acción es la respuesta visible del agente dentro del juego. Según la decisión tomada, el dron puede:

- Modificar la intensidad de una zona de viento.
- Cambiar la posición de algunos anillos.
- Ajustar el movimiento de una plataforma.
- Activar una advertencia visual.
- Reorganizar obstáculos en el Nivel 2.

12.4. Aprendizaje

El aprendizaje se basa en registrar información de intentos anteriores para que el dron no actúe siempre de la misma forma. El agente no implementará aprendizaje avanzado, sino una memoria sencilla basada en tendencias.

Por ejemplo:

- Si el jugador falla muchas veces por exceso de velocidad, el dron reduce temporalmente zonas de aceleración.
- Si el jugador evita siempre los obstáculos por el mismo lado, el dron puede mover un obstáculo hacia esa zona.
- Si el jugador mejora su puntaje, el dron aumenta ligeramente la dificultad.

12.5. Flujo general del agente

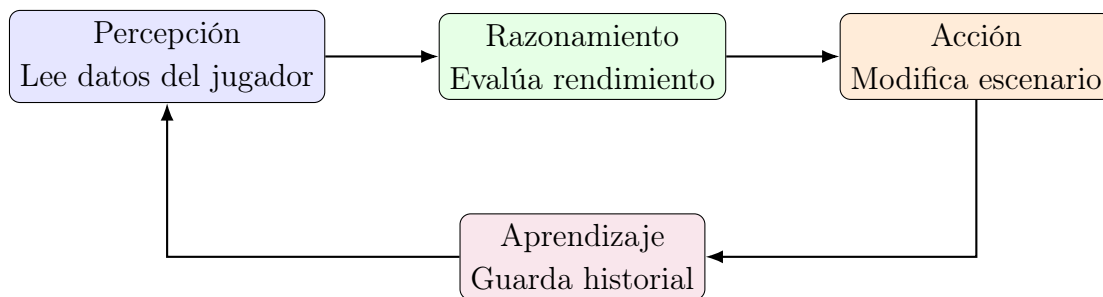


Figura 6: Flujo de funcionamiento del agente inteligente.

13. Manejo de memoria dinámica

El proyecto hará uso de memoria dinámica en los casos donde sea necesario crear objetos durante la ejecución, especialmente niveles, entidades, físicas y componentes del agente.

Ejemplos de objetos que podrían crearse dinámicamente:

- Niveles del juego.
- Obstáculos generados durante el Nivel 2.
- Físicas asociadas a cada nivel.
- Componentes internos del agente inteligente.

Definiendo así qué clase es responsable de liberar la memoria.

14. Conclusiones

El Momento II permitió transformar la idea general de *Clavados en Ciudad Academia* en una propuesta de diseño más concreta. Se definieron con mayor precisión las vistas de los niveles, las interacciones principales, las físicas parametrizables, los sprites requeridos, la estructura de clases y el funcionamiento del agente inteligente.

El diseño propuesto cuenta con dos niveles con dinámicas diferenciadas, modelos físicos distintos al movimiento rectilíneo simple, herencia propia, contenedores STL, memoria dinámica, agente autónomo y una base clara para la futura implementación con Qt.

La arquitectura planteada permite separar la lógica del juego de la interfaz gráfica, lo cual promueve una mejor organización del código y un mejor desarrollo del juego.